

Checkpoint Fundamentals of Testing

1. Testing shows the presence, not the absence, of defects.

The form clearly shows error messages like:

- Full Name - "Please fill out this field."
- Email - "Please input '@.' in the Email address."
- Password - "Password must be at least 6 characters."
- Confirm Password - "Passwords must match."

This shows that defects are present, the system is finding and showing defects

2. Exhaustive Testing Is Impossible

While the page shows some validation errors, we cannot test all possible email formats, all possible password mistakes, or all combination inputs, which would take forever. That's why the focus is on key error conditions (required fields, format, length).

This implies that you cannot test everything; you prioritize what matters most.

3. Early Testing Saves Time and Money

These validations happen as soon as I (user) interact with the form by clicking the '**Register**' button and inputting an **invalid email** and a **very short password length**. Detecting a missing name or an invalid email early reduces later feedback

Defects found early (like in validation) are cheaper to fix than those found later in the process.

4. Defects Cluster Together

Most of the defects shown are clustered around **user input validation**, especially around the password (Password and Confirm Password) criteria. When I entered a short password for the Password field and a long password length for confirm password field after filling in all input and then clicking Register, I got two error/validation messages. I.e. '**Password must be at least 6 characters**' and '**Passwords must match.**' This suggests that a small part of the software contains most defects, and the password logic is a hot spot for defects.

5. Test wear-out.

When I only test the same validations (“invalid email”, “password too short”, etc.) over and over with the same inputs, I stop finding new errors, even though other defect types might still be there. Test cases need regular review and updating because repeated identical testing stops finding new defects over time.

6. Testing Is Context-Dependent

This is a user registration form; its testing needs differ from, say, testing a flight-booking system. Here, we focus on **validation**, **security**, **usability**, and business specific to a registration context. Testing must be adapted to the software’s context and purpose.

7. Absence-of-Errors Fallacy

Even if this form shows no errors for a moment (e.g., all fields filled correctly), that doesn’t guarantee the overall system works or meets user expectations. Other issues like security, performance, and accessibility might still exist or emerge beyond these immediate validations. Finding no defects does not mean the product meets users’ needs.